

IP6: Troubleshooting network problems with an AI driven multi-agent system

P6 Project

Windisch, July 20, 2026

⇒ Titel alles
gross geschrieben

Student

Dominik Krebs

Experts

Christopher Scherb / Luc Heitz

Project Number

IMVS12

University of Applied Sciences Northwestern Switzerland

Abstract

12 yr fault?

Troubleshooting computer networks is a complex, mostly manual activity. A network operator works from the first symptom through device after device until a root cause is found. This project investigates whether an AI-driven multi-agent system can take over this troubleshooting work and at what benefit and cost. A system was built in which an orchestrator agent splits a troubleshooting request and delegates the tasks to specialised agents, all operating against live network devices.

The system was evaluated in a containerlab environment of Nokia SR Linux devices using ten reproducible scenarios (a healthy baseline plus nine injected faults across three difficulty tiers) and six language models from three price tiers. Each of the 300 runs (10 scenarios $\times$ 6 models $\times$ 5 repetitions) was evaluated manually against the scenario's known root cause.

The results show that the outcome depends mainly on the model used. With identical agent logic, the success rate spans 16% to 100%. The high-tier models diagnosed 98 of 100 runs correctly across all difficulty tiers. They typically delivered a diagnosis in roughly two to three minutes, almost independent of fault difficulty. Mid-tier models (74–82% overall) were reliable on easy faults but failed on harder ones, mostly by hitting the time cap rather than by giving a wrong result. Low-tier models were unusable and one of them repeatedly invented faults in a healthy network. With the mid- and high-tier models, a correct diagnosis cost \$0.21–\$2.91 in API fees, a fraction of the network operator time it replaces. Within the limits of a small single-fault lab, the experiments show that autonomous multi-agent network troubleshooting works today when a sufficiently capable model is used.

⇒ .. and a fraction...

Contents

List of Figures	vi
List of Tables	vi
1 Introduction	1
1.1 Project Goal	1
1.2 Research Questions	1
1.3 Outline	1
2 Background	2
2.1 Agents and AI Agents	2
2.1.1 Agent	2
2.1.2 AI Agent	2
2.2 Large Language Models (LLMs)	2
2.3 Tokens, Context Windows and Cost	2
2.4 Multi-Agent (AI) Systems	3
2.5 Function Calling	3
2.5.1 Function Calling Process Example Using the OpenAI API	3
2.6 Agent Loop and Agent Harness	5
2.6.1 Agent Loop	5
2.6.2 Agent Harness	6
2.7 Model Context Protocol (MCP)	7
3 Concepts	8
3.1 Scope: Two Independent Parts	8
3.2 Overall Architecture	8
3.3 Agent Roles and Responsibilities	9
3.4 Tool and Capability Layer	9
3.5 Model-Tier Concept	10
3.6 Evaluation Concept	10
3.7 Use Case: Automatic Syslog Investigation	10

4	Implementation	12
4.1	Agent Framework and Harness	12
4.2	Model Access	12
4.3	Delegation as Tool Calls	12
4.4	Tool Servers and Device Access	13
4.5	Context Management	13
4.6	Autonomous Investigations and Handover	14
4.7	Interfaces and Instrumentation	14
5	Methodology <i>nicht nur auch teil eval aber oh</i>	15
5.1	Experimental Environment	15
5.2	Fault Scenarios and Difficulty Levels	15
5.2.1	Difficulty Criteria	15
5.2.2	Scenario Catalogue	16
5.2.3	Scenario Descriptions	16
5.3	Experiments	17
5.4	Experiment Execution	18
5.5	Recorded Metrics	18
5.6	Evaluation Procedure	19
5.7	Aggregation	19
6	Results <i>normale weise sind results teil der evaluation gefolgt von einer discussion</i>	20
6.1	Correctness	20
6.2	Time	22
6.3	Cost and Effort	23
7	Evaluation	27
7.1	Success Rate by Difficulty	27
7.2	Time to Diagnosis	27
7.3	Cost	28
7.4	Influence of the Model	29
7.5	Limitations	30
8	Conclusion and Future Work	31
8.1	Conclusion	31
8.2	Future Work	32

9	Auxiliary Tools	33
	References	34
	Declaration of Authenticity	36
A	Appendix	37
A.1	Model Prices During the Experiment Window	37
A.2	Additional Experiment Figures	37

List of Figures

2.1	Function calling procedure [11]	4
2.2	The agent loop [14]	5
2.3	Anatomy of an agent: the model surrounded by the harness [16]	6
2.4	MCP architecture [17]	7
3.1	Architecture of the multi-agent troubleshooting system.	9
5.1	Baseline test-lab topology.	15
6.1	Found rate per scenario and model.	20
6.2	Wall-clock time to a correct diagnosis per model.	22
6.3	Mean LLM inference time per run (summed over all round-trips) and mean wall-clock time per run.	23
6.4	Mean cost per run against found rate.	24
6.5	Mean normalised input and output tokens per run.	25
6.6	Mean number of tool calls per run.	26
A.1	Malformed device commands per scenario and model.	37
A.2	Wall-clock time to a correct diagnosis per scenario and model.	38
A.3	Wall-clock time per run over all 300 runs, including wrong diagnoses and DNFs. Boxes show median and interquartile range. Dots are individual runs. The clusters at 600s are the 55 runs stopped at the wall-clock cap.	38

List of Tables

5.1	Fault scenarios grouped by difficulty tier.	16
6.1	Correct diagnoses per difficulty tier and model.	20
6.2	Per-model summary. Found rate, cost/run and DNF count are over all 50 runs. Median time to correct is over the correct, finished runs only. Cost/correct is the total cost of all 50 runs divided by the number of correct results.	23
A.1	OpenRouter list prices per million tokens during the experiment window.	37

→ sehr gut, das heutz, ist noch common
network immer benutzen

1 Introduction

Troubleshooting computer networks is a time-consuming and still largely manual activity. When a service reports a problem or a host loses connectivity, an engineer needs to consult the available tools and check one device after another, running show commands, inspecting interface counters, routing tables, ARP caches and syslog messages until a fault is localised. The path from the first symptom to the true root cause often requires many steps and much time. As networks grow in size and complexity, this diagnostic effort translates directly into operational cost and a longer mean time to repair (MTTR).

Recent advances in large language models (LLMs) and, in particular, in agentic systems that let an LLM call external tools, observe the results and decide on the next action, suggest a new way to support this work. Instead of a single-agent design, a multi-agent system can split the problem into smaller parts and assign them to specialised agents. Whether such a system can be used to assist in troubleshooting or even find problems autonomously is an open and practically relevant question.

1.1 Project Goal

The goal of this project is to design and build an AI-driven multi-agent system for network troubleshooting and to use it as an instrument to evaluate *which kinds of network problems can realistically be troubleshot by such a system, and at what benefit and cost.*

1.2 Research Questions

The project is guided by five research questions:

1. **Time:** How much time can be saved in troubleshooting by using the agent network compared to a manual workflow?
2. **Difficulty:** Problems of which difficulty level can be troubleshot by the agent network?
3. **Success rate:** What is the success rate for the correct analysis of the problems at each difficulty level?
4. **Cost:** How much can potentially be saved or, conversely, how much does it cost to use an agent network for troubleshooting network issues?
5. **Model influence:** How much do the success rate and the cost of the agent network depend on the choice of model?

1.3 Outline

Section 2 introduces the underlying technologies: LLMs, AI agents, function calling, the agent loop and the Model Context Protocol. Section 3 explains how these building blocks are combined into the multi-agent troubleshooting system and section 4 describes its realisation. Section 5 defines the fault scenarios, the difficulty tiers, the evaluated models and the experiment procedure. Section 6 reports the measurements, section 7 interprets them with respect to the research questions and discusses the limitations, and section 8 concludes the report and outlines future work.

↳ 1. Erst statt

↳ 1. und 1. Teil 2
in der section / fig
etc

2 Background

This section gives the necessary background information about the technologies used in this work.

2.1 Agents and AI Agents

This subsection first defines the traditional term *Agent* and then the newer *AI Agent*.

2.1.1 Agent

From a very basic perspective, an agent is an entity which is able to produce some effect or make changes in its environment [1].

2.1.2 AI Agent

With the rise of generative AI a new term “AI Agent” has emerged. An AI Agent is a piece of software that works towards a goal it has been given. It takes in information from its environment (structured or not), reasons about what it sees and then acts to get closer to that goal. A traditional agent executes a predetermined, deterministic sequence of steps, whereas an AI agent responds to what it observes while running, adjusting its behaviour and the results it produces as the situation changes. At its core, the decisions of such an agent are made by an LLM (see section 2.2) [2].

2.2 Large Language Models (LLMs)

An LLM is a statistical model that predicts the likelihood of the next token in a sequence of text. Trained on very large text corpora, largely taken from the public internet, it learns to produce plausible continuations for a given input prompt [3].

2.3 Tokens, Context Windows and Cost

LLMs do not operate on text (words and characters) directly. Text has to first go through a *tokenizer* that breaks it into pieces called *tokens* which can be words, sub-words or even just single characters. The goal of a *tokenizer* is to return an efficient set of tokens to represent a text dataset [4].

Each model can only process a limited number of tokens at once. This upper bound is referred to as the *context window* and covers the prompt, any provided tools, the conversation history and the generated response. Beyond being a hard limit, longer contexts also degrade the quality of the model’s output: Liu et al. [5] show that models use information most reliably when it appears at the beginning or the end of the context and perform noticeably worse when relevant information is located in the middle, a phenomenon known as “lost in the middle”. The degradation also depends on *how much* context surrounds the relevant information, not only on *where* it sits: Hong et al. [6] evaluate a range of state-of-the-art models on deliberately simple tasks and find that performance declines as the input grows longer, even when the task itself does not become harder. These two properties, the finite context window and the degradation over long contexts, are a central motivation for distributing work across several specialised agents rather than handling everything in a single, ever-growing context (see section 2.4).

Zusammen fassen
subjektive
2

einsetzen 2.5 das heute die meisten
AI Agents auf LLMs basieren

+

2.4 Multi-Agent (AI) Systems

A multi-agent (AI) system (MAS) is a system composed of multiple interacting agents. A MAS is able to solve more difficult problems than an individual agent (a monolithic system) [7].

With the rise of LLM-based agents (AI agents) (see section 2.2), this idea has been transferred to systems in which the individual agents are driven by LLMs. The survey by Tran et al. [7] describes how multiple LLM agents collaborate through different coordination structures. The structure and collaboration strategy chosen for this work are described in section 3.

2.5 Function Calling

LLMs have great generative capabilities but they lack the ability to perform any task beyond generating a response. This means LLMs are not able to retrieve any external information in real time. Everything the model knows comes from its static training data, so an answer can sound convincing while actually being wrong or simply out of date, a behaviour commonly called hallucination [8]. To address this limitation a paradigm named “Function Calling” has been developed. It mitigates these problems by giving LLMs the ability to interact with external systems by invoking predefined blocks of code (functions), so that answers can be grounded in fresh, retrieved information instead of training data alone [9].

2.5.1 Function Calling Process Example Using the OpenAI API

Figure 2.1 illustrates the function calling principle using the OpenAI function calling API as an example. The function calling API is an optional part of the new *Responses API* or the older *Chat Completions API* [10, 11].

1. The prompt message and all the tool definitions are sent to the model.
2. Based on the provided prompt and tools the model decides to call some function, providing the necessary arguments (in this example “paris”).
3. The function chosen by the model is executed.
4. All prior messages together with the result (output) of the function execution are sent back to the model.
5. Based on all the messages the model gives back a final response [11].

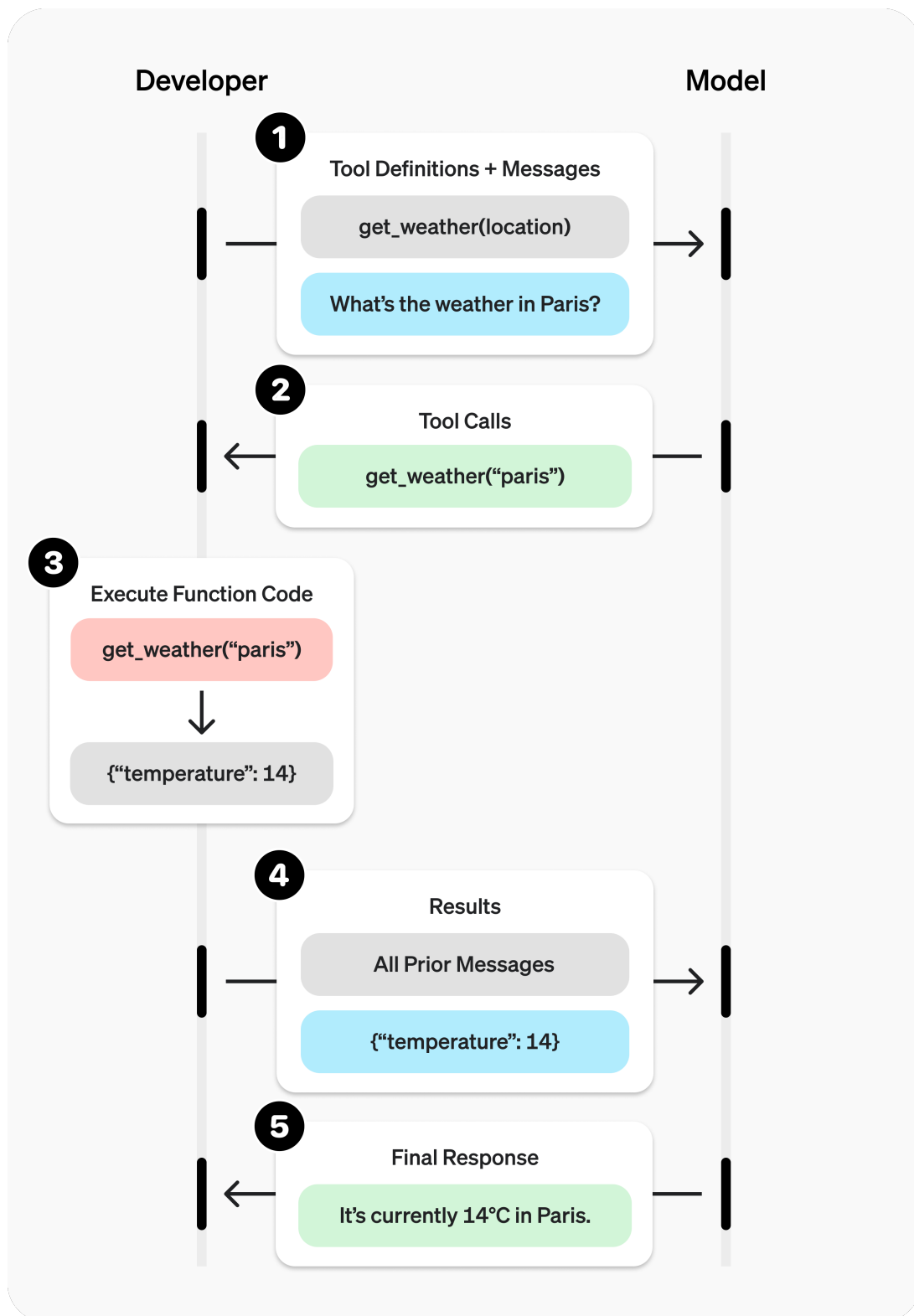


Figure 2.1: Function calling procedure [11]

2.6 Agent Loop and Agent Harness

name for the
headers

2.6.1 Agent Loop

Function calling (see section 2.5) describes a *single* exchange in which the model receives a function, requests a function execution and receives its result. On its own, this does not yet make a system agentic. The model still needs to be invoked repeatedly, decide when another tool call is necessary, and recognise when the task is complete. This repetition is captured by the term *agent loop*, the core control cycle of an AI agent. The model receives the current context, decides on an action (either calling a tool or producing a final answer), the chosen tool is executed, its result is appended to the context, and the cycle repeats until the model returns a final answer or a stop condition is reached [12, 13, 14]. These steps of reasoning and acting were introduced as the *ReAct* pattern by Yao et al. [12]. The agent loop is what turns isolated function calls into the adaptive, goal-oriented behaviour of an AI agent as introduced in section 2.1.2.

Figure 2.2 shows this cycle: an initial input and context enter the loop, the model reasons and selects a tool, the tool is executed and its result is fed back for the next round of reasoning, and the loop repeats until the model produces a final response.

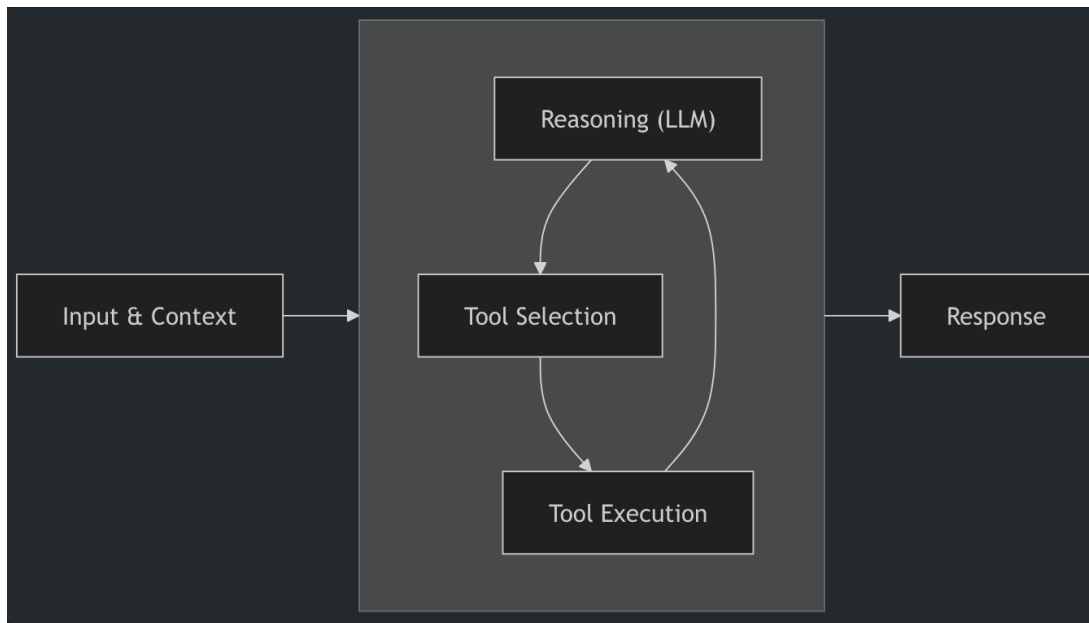


Figure 2.2: The agent loop [14]

2.6.2 Agent Harness

An LLM by itself cannot run this loop (the agent loop), as it only predicts a single continuation for a given input (see [section 2.5](#)). The loop, the dispatching of tool calls, the accumulation of the message history and the evaluation of stop conditions all have to be implemented by the surrounding software. This runtime software is commonly referred to as the *agent harness*: everything that is built around the model so that it can actually work on a task instead of just answering a single prompt, often summarised as $Agent = Model + Harness$ [15, 16]. The harness is the engine that drives the agent loop (the core) and manages the things around it. It forwards the conversation and the available tool definitions to the model, executes the tool calls the model requests and maintains the message history across iterations. In practice a harness is usually provided by an *agent framework* with APIs to modify and extend its functionality. The concrete framework used in this work is described in [section 4](#).

Figure 2.3 illustrates this relationship: the model sits at the centre and only reasons and decides, while everything around it, the context injection, control, action (tool calls), observation and persistence, is provided by the surrounding harness.

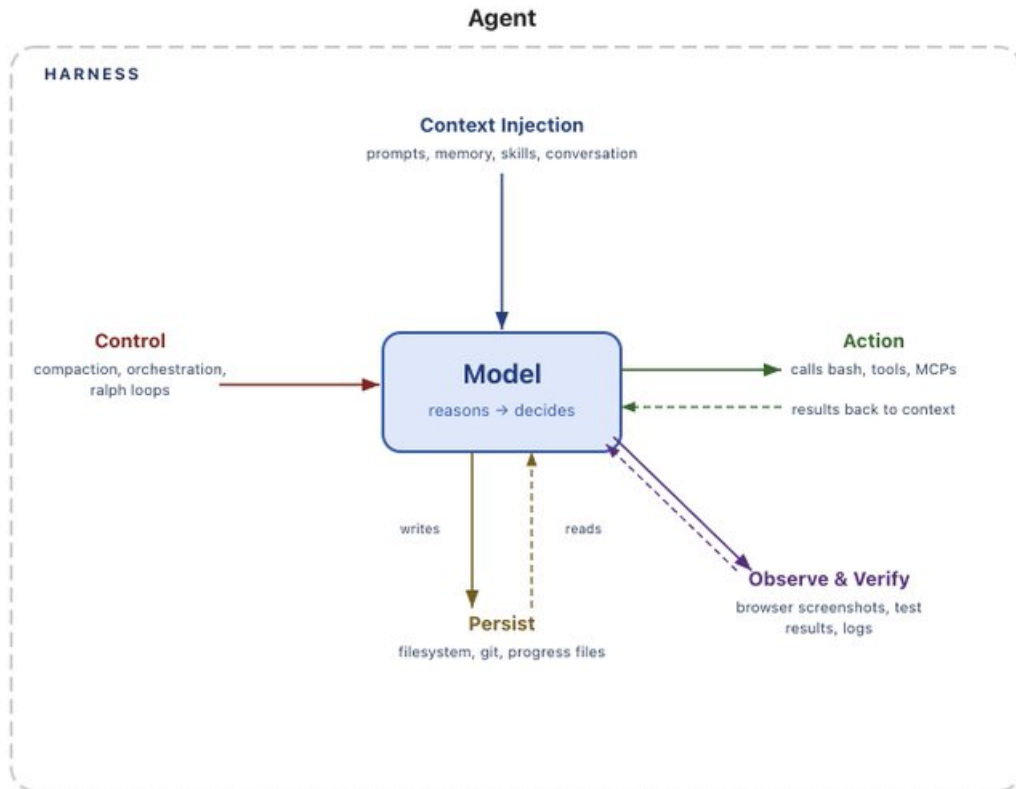


Figure 2.3: Anatomy of an agent: the model surrounded by the harness [16]

2.7 Model Context Protocol (MCP)

The Model Context Protocol (MCP) is an open standard that specifies how AI applications communicate with external systems [17]. Through this protocol, an AI application (for example a chat assistant) gains structured access to the outside world: it can read from data sources such as databases or local files, and it can invoke executable tools.

The core problem MCP addresses is the fragmentation of AI integrations. Before MCP, hooking an AI application up to a new data source meant writing yet another one-off integration, an approach that quickly stops scaling as the number of connected systems grows [18]. This is commonly framed as the $N \times M$ integration problem [19]. Without a common standard, connecting N AI applications to M external systems (data sources and tools) requires a custom integration for every AI application–external system pair, resulting in up to $N \times M$ separate integrations that must each be built and maintained individually. MCP replaces this with a single shared protocol. Every AI application implements the MCP client side once, and every external system is exposed through an MCP server once. This reduces the integration effort from a multiplicative $N \times M$ to an additive $N + M$ [19].

As illustrated in fig. 2.4, MCP functions similarly to a “USB-C port for AI applications” [17], one standard interface through which an AI application can reach many different external systems.

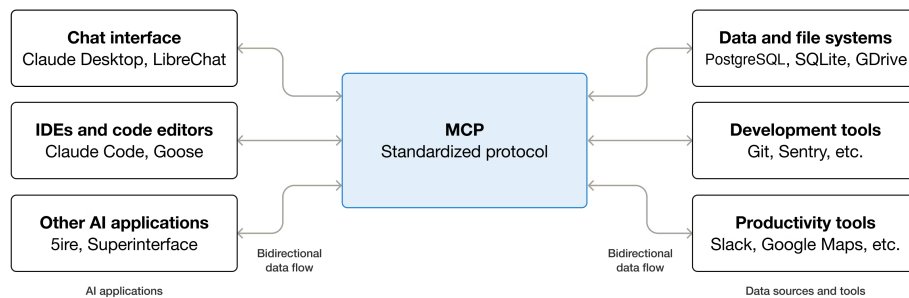


Figure 2.4: MCP architecture [17]

3 Concepts

While section 2 introduced the underlying technologies in general terms, the concepts section explains how they are applied to the problem of network troubleshooting and why the multi-agent troubleshooting system is designed the way it is. The realisation (implementation) in code is deferred to section 4, and the experimental procedure used to evaluate the design is described in section 5.

3.1 Scope: Two Independent Parts

The work described in this report consists of two distinct and largely independent parts, which it is important to keep apart conceptually.

The first part is the core of the project and addresses the research questions stated in section 1, which network problems can be troubleshot by a multi-agent system, with what success rate across difficulty levels and at what cost. This part is driven by a human operator who poses a troubleshooting goal to the orchestrator agent and it is evaluated against multiple fault scenarios (see sections 3.5 and 3.6).

The second part is an independent feasibility use case rather than a measured research question, the *automatic syslog investigation*. The question here is simply whether it is possible to build a system that reacts autonomously to certain syslog messages and starts a self-contained initial investigation without any human trigger. A second question is how such a mechanism works in practice and how a human operator can take over and continue the troubleshooting without repeating the work the autonomous investigation has already done. This use case reuses the same agent building blocks shown in fig. 3.1 but uses a different entry point (`investigation_orchestrator`). It is described on its own in section 3.7 and is not part of the quantitative evaluation.

3.2 Overall Architecture

The system is structured as a *hierarchical, role-based* multi-agent system, the collaboration pattern introduced in section 2.4. The *orchestrator* agent receives a troubleshooting goal and delegates sub-tasks to specialised sub-agents, each responsible for a specific task or domain (see section 3.3).

The main reason for distributing the work across several agents instead of using one monolithic agent comes from the limitations discussed in section 2.3. A single agent would have to hold the entire troubleshooting context (all tool definitions, all device output, the full conversation history) in one large context window. This approach risks hitting the context limit quickly and thereby degrading output quality through the “lost in the middle” effect (introduced in section 2.3). In comparison, when using multiple specialised agents, they receive only focused instructions and tools, which keeps the individual contexts small and reduces context-related quality degradation.

Each LLM-driven agent (AI agent) in this architecture is an independent agent loop (see section 2.6). It runs its own loop over its own context and tools, using the same underlying agent harness. (The topology agent is the one exception, section 4.3 explains why it is not LLM-driven.) Delegation is itself realised as a tool call. The orchestrator’s loop invokes a sub-agent as a tool, the sub-agent runs its own loop to completion and returns its result, which the orchestrator then incorporates into its own context.

Figure 3.1 shows the resulting architecture, the orchestrator at the top delegates sub-tasks to the specialised sub-agents, each of which has its own tools.

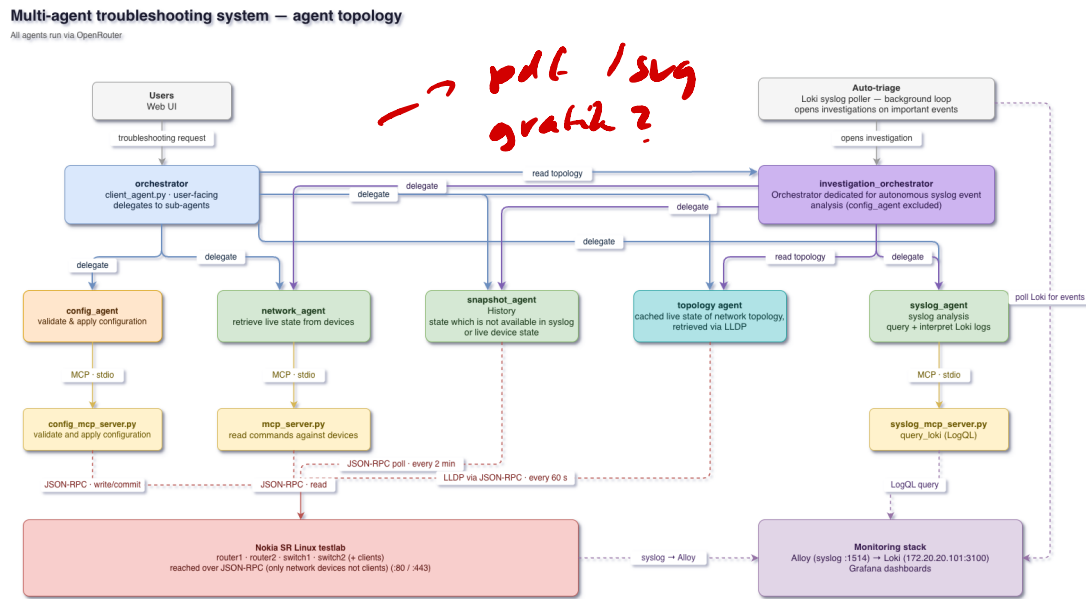


Figure 3.1: Architecture of the multi-agent troubleshooting system.

3.3 Agent Roles and Responsibilities

Each agent has one responsibility and a fixed, narrow scope. The following roles make up the system:

- **Orchestrator (user-facing):** Receives user input and coordinates the sub-agents. It splits the user’s goal (task) into sub-tasks and delegates them to the sub-agents and combines their results into a final answer. Since the sub-agents are “stateless” the orchestrator needs to supply all the required information to the sub-agents.
- **Investigation orchestrator (autonomous):** Coordinates the autonomous syslog-event-based investigations. It runs the same delegation pattern as the user-facing orchestrator without access to the configuration agent.
- **Network agent:** Retrieves live device state through read commands.
- **State-snapshot agent:** Provides historical device state. The state-snapshot agent focuses on information which cannot be retrieved via syslog or device live state but can be useful for troubleshooting (like past ARP entries or routing tables).
- **Topology agent:** Supplies the (cached) network topology so other agents can reason about how devices are connected. It compares the “should be” state of the topology with the actual “live” state of the network (via LLDP queries).
- **Syslog agent:** Queries and analyses syslog data for the entire network.
- **Configuration agent:** The only agent with *write* access. It is able to apply configuration changes to devices.

3.4 Tool and Capability Layer

Agents gain the ability to interact with the outside environment through the function-calling mechanism described in section 2.5, exposed via the Model Context Protocol introduced in section 2.7. Most agents are served by their own MCP server, so every agent only gets the tools that are required for its specific task.

This split of expertise is a design choice that supports the architecture of section 3.2. It keeps every agent's tool definitions, and therefore its context, small and precise.

Not every agent follows this pattern. Two agents, the topology agent and the state-snapshot agent, do not use MCP servers. Their tools are defined directly inside the agent framework as native function tools, because both read state that a background task inside the same process already collects and caches (topology and device state). Sending such calls through an MCP server would add a process boundary and serialisation overhead without any of the benefits described in section 2.7. The topology agent goes one step further. As section 4.3 describes, it has no model and no agent loop at all. The state-snapshot agent is a regular Pydantic AI agent. Only its tools are in-process. 

3.5 Model-Tier Concept

LLMs can be compared to each other using a variety of metrics. This work uses the *price per million tokens* as the comparison metric, because cost is a primary research question. Models are grouped into *tiers* according to their *price* (low, mid and high). The system is designed so that the model tier can be changed independently of the agent logic, which makes it possible to study how model choice affects the success rate and cost in terms of troubleshooting networking issues. The concrete models assigned to each tier are described in section 5.

3.6 Evaluation Concept

To answer which problem difficulties can be troubleshoot and at what success rate, the system is evaluated against different fault scenarios. Each scenario injects a specific, known fault into the network (for example a downed interface, a missing VLAN on a trunk, or a silently dropping  ACL) and is assigned a *difficulty level*. The difficulty level conceptually captures how much correlation across domains (topology, live state, history, syslog) and how many steps are required to localise the fault.

3.7 Use Case: Automatic Syslog Investigation

As mentioned in section 3.1, this use case is a separate, exploratory part of the project. Its purpose is to answer whether an autonomous, event-driven investigation is possible at all and how it can be integrated into the network operator's workflow. It builds on the autonomous investigation orchestrator introduced in section 3.3.

Autonomous reaction to syslog events

The first question is whether the system can react to the network on its own. Instead of waiting for a human to describe a symptom, the system observes the incoming syslog events and, when a message matches the configured pattern, an automatic investigation is launched. The autonomous investigation orchestrator then follows the same pattern as the user-facing orchestrator with the exception of not having access to the configuration agent. It is also designed so investigations cannot be opened recursively. The result is a self-contained initial report produced from a syslog event without any human involvement.

Handover without duplicating work

The second question concerns what happens after an autonomous investigation has run. Such an autonomous report may be incomplete and need further investigation by a human (network operator). The operator must therefore be able to take over and continue troubleshooting from the point the autonomous investigation reached, without repeating the work it already did. The concept addresses this by keeping the autonomous investigation's findings as reusable context

(with a unique ID). When the operator resumes the case, this context is fed back into the same investigation orchestrator that produced it, so the case continues from its existing state instead of starting from scratch. How this handover is realised concretely is described in section 4.

4 Implementation

The implementation of the design in section 3 is described in this chapter. The full source is available in the project repository¹.

4.1 Agent Framework and Harness

The agent harness introduced conceptually in section 2.6 is provided in this work by *Pydantic AI*, a Python agent framework [20]. Pydantic AI implements the agent loop, does the function-calling exchange with the model, manages the message history across iterations and exposes the tools of each agent.

Every LLM-driven agent of section 3.3 is one Pydantic AI agent, defined by four things, a name, an instruction text (its system prompt), a model and tools. The only exception is the topology agent, which has no model and is described separately below. All agents run in a single Python process. Agents are plain objects that call each other directly. There is no network protocol or separate service per agent, only the MCP tool servers of section 4.4 run as separate subprocesses.

At the start of the project, the plan was to use the A2A protocol² for communication between the agents. This would have allowed the agents to run in separate processes or even on different machines. A2A was dropped because it would have added additional complexity, without helping answer the project's research questions.

4.2 Model Access

All models are accessed through *OpenRouter*, a gateway that exposes models from different vendors behind one API [21]. Switching the model an agent uses only means changing a model identifier (a string). This is what makes the experiment matrix of section 5.3 practical, because the model can be changed without touching the agent logic. *altering / adjusting*

Two settings are applied to every agent. First, the reasoning effort is fixed to the same level ("high") for all models, so that model versus model comparisons are not influenced by each provider picking its own default reasoning depth. Second, usage accounting is enabled on every request, so each model response carries its usage information. This usage information is the basis for the cost and token metrics of section 5.5.

4.3 Delegation as Tool Calls

Delegation between the orchestrator and its sub-agents (section 3.2) is realised as ordinary function calling. Each sub-agent is registered on the orchestrator as a tool. The tool description tells the orchestrator's model what the sub-agent can do and when to use it. When the model calls the tool, the wrapper runs the sub-agent's own agent loop to completion and returns its final answer as the tool result.

Sub-agents are stateless. They never see the orchestrator's conversation and keep no conversation history between calls. The orchestrator's instructions therefore require every delegated request to be self-contained, carrying the user's goal, the relevant details and any important information from earlier sub-agent calls. Read-only sub-agents can be called in parallel when a request involves independent lookups (the decision is up to the LLM).

One agent is not based on an LLM making decisions. The topology agent is implemented as a background task that queries the LLDP neighbours of every device once a minute. Then it

¹<https://github.com/FHNW-Security-Lab-Student-Projects/ip6-autonome-netzwerkadministration>

²Agent2Agent (A2A), an open protocol for communication between AI agents, <https://a2a-protocol.org/>.

work it is future work
nehmen

compares the “live” state against the “should be” topology definition of the lab and caches the result. The orchestrator’s topology tool simply returns this cache (so the response is faster). Since topology retrieval is deterministic and follows a strict order of events, an LLM would just add cost without adding anything beneficial to the agent.

4.4 Tool Servers and Device Access

Following the design of [section 3.4](#), each agent using tools via MCP is served by its own MCP server ([section 2.7](#)), implemented with *FastMCP* [22] and started as a subprocess of the main process.

All interaction with the lab devices goes through the JSON-RPC interface of Nokia SR Linux [23]. The tools offer two routes to device data. The first is executing formatted show commands and the second is reading a specific YANG path directly from the state or running-configuration datastore. As an attempt to keep the models from using invalid commands, a reference of valid SR Linux read commands is added in the network agent’s system prompt. When a device still rejects a command, the failure is detected in the tool output and recorded. This is the source of the invalid-command count of [section 5.5](#).

For syslog, all devices log to a central *Loki* instance [24], which the syslog agent queries through its MCP tool. A practical challenge here is time. An LLM has no reliable clock and its prompt may be minutes old by the time a tool runs. The tools accept time values such as “now”, a relative offset (“15m ago”) or an absolute timestamp, and resolve them against the real clock at tool runtime (so the time of the device where the tool code runs is used), so the model never has to compute a timestamp itself.

4.5 Context Management

Device command output is the biggest factor in terms of context growth in this project. On every iteration of an agent loop, the full message history is sent again, so large tool returns are paid for again on every subsequent step (without prompt caching). Two actions were taken to address this, both following from [section 2.3](#).

The first acts at the source. Tool responses are cut off at a fixed size of 15 000 characters. A truncated response ends with a note telling the model to retry with a more narrowly scoped query (for example to query interfaces directly rather than doing summary queries).

The second acts on the history. Before sending a request to the model, a processor estimates the current prompt size based on the token count the provider reported for the previous request. Only when the estimate approaches the model’s context window (whose size is looked up per model, since the models differ) does it perform the operations. The oldest oversized tool returns are each replaced by a sentence stating which tool was called and that it can be re-invoked if the output is needed again. The most recent returns and all error messages are kept unchanged, because an error message tells the model what went wrong and how to avoid repeating the same mistake. This keeps long runs inside the context window and reduces the “lost in the middle” effect, while still letting the model re-fetch anything it needs.

4.6 Autonomous Investigations and Handover

The syslog use case of [section 3.7](#) is driven by a background task that polls Loki every 30 seconds. A log event matching the configured severity opens an investigation, keyed by device and a normalised message so that repeating log lines do not open multiple investigations for the same type of log message. The investigation is then run as a background task via the investigation orchestrator agent (“`investigation_orchestrator`” in Figure 3.1). Because this orchestrator lacks the tools to open or continue investigations (only the user-facing orchestrator agent has them), an investigation can never trigger further investigations recursively.

Each investigation is persisted to disk as a record containing the complete serialised message history of the agent run. This history is what realises the handover described in [section 3.7](#), the user-facing orchestrator has tools to list, inspect and continue investigations and continuing a case loads the stored history and feeds it back to the investigation orchestrator, which is the agent that produced it ([section 3.3](#)). The model therefore still holds all earlier findings and tool results and continues from where the autonomous run stopped, instead of re-running the same commands. The user is also able to ask questions about previous investigation steps.

4.7 Interfaces and Instrumentation

The system has two entry points, a web chat interface with a model selector (built on the framework’s bundled web UI), and a status page showing all open investigations. In the interactive interfaces the orchestrator keeps its message history across turns, so a troubleshooting session is one continuous conversation.

All agent runs, tool calls and device requests are traced with *Logfire* [25]. The traces show, per run, which sub-agents were called with which requests and what each tool returned. This was the main debugging instrument during development.

The experiments of [section 5](#) reuse the interactive system unchanged, except that the configuration agent and the investigation tools are disabled through feature flags, so the measured system is exactly the read-only troubleshooting core.

5 Methodology

The system designed in section 3 is evaluated experimentally. Where section 3.6 defined what a scenario and a difficulty level are, the following describes how the experiments are run.

5.1 Experimental Environment

All experiments are run against a virtual network using *containerlab* [26], so that the agents interact with real device behaviour. The lab consists of Nokia SR Linux [27] nodes with a predefined configuration and topology in which the test clients can reach each other (fig. 5.1). This means the baseline is a fully functional network.

The topology contains four SR Linux nodes and four Linux clients. On one side, switch1 connects client1 (VLAN 10, 192.168.1.0/24) and client2 (VLAN 20, 192.168.10.0/24) and carries both VLANs over a trunk to router1, which acts as their gateway. On the other side, switch2 connects client3 and client4, which share VLAN 30 (10.10.10.0/24) with router2 as their gateway. router1 (AS 65001) and router2 (AS 65002) exchange the client prefixes over an eBGP session on the link between them. All devices send their syslog messages to a central log store (see section 4.4).

containerlab 'testlab' — network topology

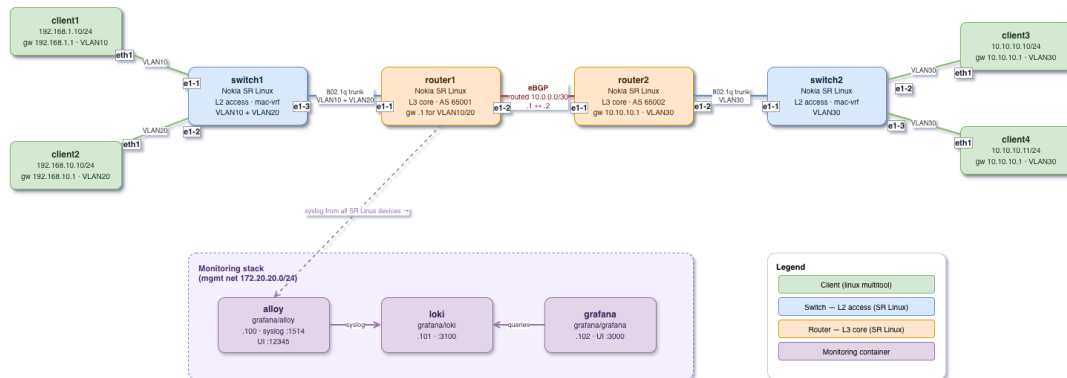


Figure 5.1: Baseline test-lab topology.

5.2 Fault Scenarios and Difficulty Levels

The system is evaluated against a set of fault scenarios. Each scenario is packaged as a folder. Its setup script injects exactly one known fault into the healthy baseline configuration (topology). A scenario consists of:

- **setup.sh**: Injects the fault before the queries are sent.
- **teardown.sh**: Restores the baseline after the experiment is done.
- **queries.txt**: The open-ended troubleshooting prompt given to the orchestrator (for example “investigate the root cause”), giving no hint about where to look.
- **ground_truth.yaml**: A description of the true fault (root cause), used as the reference during manual evaluation (section 5.6).

5.2.1 Difficulty Criteria

Difficulty is defined by how far an agent has to go from the obvious first check to reach the root cause. A fault being visible in syslog does not by itself make a fault easy. Three properties

determine the difficulty: (1) whether the obvious signal tells the truth or is misleading (for example a link that is up while BGP is down), (2) how many devices or layers lie between where the symptom shows up and where the fault actually sits and (3) how many signals must be correlated (a single show command versus comparing two route tables or varying a packet size).

5.2.2 Scenario Catalogue

Table 5.1 lists the scenarios grouped by difficulty tier. A baseline scenario with no fault is included to verify that the system does not invent problems in a healthy network.

Scenario	Tier	In syslog?	What localises it
<code>basic-client-communication</code>	baseline	—	confirm full reachability, invent no fault
<code>intf-down</code>	easy	yes	one <code>show interface</code>
<code>client3-port-down</code>	easy	yes	one <code>show interface</code>
<code>switch1-uplink-down</code>	easy	yes	one <code>show interface</code>
<code>missing-export-policy</code>	medium	no	RIB-out (session state lies)
<code>missing-vlan-on-trunk</code>	medium	no	trunk VLAN membership
<code>duplicate-ip-arp</code>	medium	yes	syslog <code>Duplicate add</code> event
<code>acl-silent-drop</code>	hard	no	ACL drop counters
<code>one-way-route-filter</code>	hard	no	route-table asymmetry (two RIBs)
<code>mtu-blackhole</code>	hard	no	packet-size probing + interface MTU

Table 5.1: Fault scenarios grouped by difficulty tier.

5.2.3 Scenario Descriptions

The following paragraphs describe each scenario in table 5.1: the fault that `setup.sh` injects and why the scenario sits in its tier according to the criteria of section 5.2.1.

Baseline

`basic-client-communication` injects no fault. The network is healthy and the query asks the agent system if `client1`, `client2` and `client3` can reach each other. The scenario checks that the system reports full reachability and does not invent a fault that is not there. This is a check against hallucinated faults.

Easy tier

In the easy scenarios the fault sits exactly where the symptom points and the obvious signal is the truth. A single `show` command on the right device reveals it, and the event is logged.

`intf-down` The interface connecting `router1` to `router2` is administratively disabled. The operational state goes down and the event appears in syslog, so one interface check on `router1` reveals the fault.

`client3-port-down` The access port on `switch2` towards `client3` is administratively disabled. The down state is visible in a single interface check and in syslog.

`switch1-uplink-down` The trunk uplink from `switch1` to `router1` is administratively disabled. The operational state goes down and the event appears in syslog, so one interface check on `switch1` or `router1` reveals the fault.

Medium tier

In the medium scenarios the obvious signal is misleading. The fault sits one step away from it, so the agent has to look one layer deeper.

- missing-export-policy** The eBGP export policy on router1 is removed. The BGP session remains established, but router1 no longer advertises its LAN prefixes, so router2 has no return route. The session state (BGP session) gives a false picture, checking it and stopping results in a miss. There is no syslog entry generated. The agent must inspect what is actually advertised and received (RIB-out/RIB-in), one layer below session state.
- missing-vlan-on-trunk** The trunk uplink from switch1 towards router1 stops carrying the VLAN of client1, leaving that VLAN isolated from the network. The two obvious checks both pass, since the trunk port stays up (it still carries the other VLAN) and client1's access port is correctly configured. There is nothing logged to syslog.
- duplicate-ip-arp** A second host claims client3's IP address while client3's switch port is briefly down, so the gateway (router2) relearns the address onto the wrong MAC and its ARP cache stays poisoned after client3 returns. A ping to the address still succeeds, because the second host answers, so a plain reachability check reports the network as fine. The giveaway is the mismatch between a pingable IP and a dead service on client3. The scenario is medium rather than hard because SR Linux names the conflict in syslog (a duplicate-address event listing both competing MACs).

Hard tier

In the hard scenarios the configuration looks complete at every layer the agent normally checks and nothing is logged. The fault hides where the agent has no reason to look until everything else is ruled out. The main risk in this tier is that the agent concludes the network is healthy.

- acl-silent-drop** An ingress ACL on router2 silently drops ICMP traffic from client1's subnet. Routing and BGP are fully healthy. The drop happens in the data plane, the agent has to think and check counters or inspect for a specific configuration. There is no syslog entry triggered when an ACL entry is hit.
- one-way-route-filter** A BGP import policy on router1 rejects client3's prefix. The session is Established and router2 still learns client1's prefix, so every single-device check passes. The fault shows up only as an asymmetry between two route tables: the agent must compare router1's RIB against router2's and notice one prefix missing on one side.
- mtu-blackhole** The IP MTU of router1's inter-router interface is lowered to 1280 bytes, creating a path-MTU blackhole. Small packets pass while large packets with the do-not-fragment flag are silently dropped. A normal ping succeeds, so the agent can wrongly conclude the network is fine. Finding the fault requires varying an input (trying with increasing packet sizes) and then inspecting the interface MTU.

A fourth, *history-required* tier was originally planned: faults that are visible only by comparing the current state against a recorded baseline, which would have made the state-snapshot agent (section 3.3) strictly necessary. Its intended use-case, **duplicate-ip-arp**, turned out to be visible in syslog on real SR Linux gear and was therefore classified as medium instead. As a consequence, no scenario in the final catalogue strictly requires the state-snapshot agent.

5.3 Experiments

To study the effect of model capability and cost (the model-tier concept of section 3.5), every scenario is run against different models taken from the three price tiers, with two models per tier:

- **High:** anthropic/claude-opus-4.8, openai/gpt-5.5
- **Mid:** z-ai/glm-5.2, qwen/qwen3.7-max
- **Low:** deepseek/deepseek-v3.2, mistralai/ministral-14b-2512

can't
find
this

The full matrix is the cartesian product of all scenarios and all models, with each combination repeated 5 times because of the non-deterministic nature of the models. The same model is used across all agents. A wall-clock cap of 600 seconds per run bounds the cost and duration of the 300-run matrix. Its influence on the results is discussed in section 7.5. A run that exceeds the cap is recorded as *did-not-finish* (DNF) while keeping the statistics of any sub-agents that already completed.

5.4 Experiment Execution

The system under test is restricted to the read-only troubleshooting core of section 3.2: the user-facing orchestrator and its sub-agents. The configuration agent and the syslog-triggered autonomous investigation (section 3.7) are disabled for all runs (section 4.7), since the experiments measure the system’s ability to find a fault. Applying a fix or detecting a fault unprompted are separate capabilities outside the scope of this evaluation.

Each run follows the same procedure to guarantee a clean, comparable starting point.

1. The lab is redeployed to a clean baseline and given time to settle (for example for BGP to come up).
2. `setup.sh` injects the scenario’s fault.
3. The orchestrator and all sub-agent tool servers are started, and the scenario’s query is sent through the agent pipeline of section 3.2 (restricted as described above).
4. Each turn of the run is appended to a structured log, together with a shared run identifier.
5. `teardown.sh` restores the baseline in a finally block, so the next run is not contaminated. Additionally the lab is redeployed between every run.

One property of this setup matters for interpreting the results. Each run gets a single, broad symptom description and no follow-up. The orchestrator receives this description and is explicitly instructed not to ask for further information. From there the system (multi-agent network) has to work autonomously to a final answer. This restriction exists because the matrix runs unattended. There is no human who could reply. Real operation would be interactive, with a network operator able to guide the investigation. Section 7.5 discusses what this restriction likely costs in success rate. The experiments measure the system in its most restricted mode of use.

Correctness for the experiments is scored separately (based on the final result of the orchestrator agent) afterwards by a human (section 5.6).

5.5 Recorded Metrics

For every run the following measurements are recorded, mapping directly onto the project’s research questions:

- **Duration** per run, in two forms. The *wall-clock duration* of the agent session is the time from submitting the query to the final answer, taken from the session’s tracing spans so that the tooling around the run (lab redeployment, cost polling) is not counted. It is the basis for the time-saving question. The *inference duration*, the summed model generation time across all agent runs as reported by OpenRouter per generation, compares model speed but is not a troubleshooting time. Agents running in parallel can make it exceed the wall clock.

- **Token usage** (prompt and completion) and **cost**, which is computed from the token counts and the model’s advertised price per million tokens on OpenRouter. These are the basis for the cost question. Prompt caching is deliberately left out of this calculation. Not every provider applies server-side caching by default and where it exists it is often best-effort, so whether a run benefits would depend on which provider happened to serve it rather than on the model being tested. Pricing every token at the full list rate keeps the comparison fair across models, at the price of being a list-price estimate rather than the billed amount. In practice, real costs would tend to be lower where caching applies.
- **Completion status** (completed vs. DNF) and **invalid command count**, recorded when an agent issues a malformed device command, as an indicator of robustness.
- **Correctness verdict** per run, assigned manually ([section 5.6](#)), the basis for the success-rate question.

5.6 Evaluation Procedure

Correctness is scored manually. There is no LLM judge. Only the **final answer of each run** is evaluated. For each run the agent’s final answer is compared to the scenario’s root cause and classified as *found* or *missed*. For the baseline scenario, correctly reporting the network as healthy counts as *found*.

The decision as to whether the root cause was correctly found is up to the human (author of this work) looking at the final answer of the agent (orchestrator).

5.7 Aggregation

The results of the evaluation are presented in [section 6](#) and used to answer the research questions across the different models in [section 7](#).

6 Results

This section reports the results from the conducted experiments of section 5.3. What they mean for the research questions is discussed in section 7. The dataset consists of 300 runs (10 scenarios $\times$ 6 models $\times$ 5 repetitions). Every run was evaluated manually (by the author of this work) against the scenario’s root cause as described in section 5.6. 55 of the 300 runs hit the 600-second wall-clock cap and were stopped (DNF), 38 for DeepSeek-V3.2, 8 each for GLM-5.2 and Qwen3.7-Max, and 1 for Claude Opus 4.8. None of them had delivered a correct final answer when stopped, so all 55 count as misses. Supplementary figures are collected in section A.2.

6.1 Correctness

Table 6.1 shows how many runs ended with a correct result, per difficulty tier and model. The share of runs with a correct result is called the *found rate* in the following. It is the success rate of the research questions (section 1.2). Figure 6.1 breaks the same data down to individual scenarios.

Model	Correct diagnoses / runs				overall
	baseline	easy	medium	hard	
claude-opus-4.8	5/5	15/15	14/15	14/15	96 %
gpt-5.5	5/5	15/15	15/15	15/15	100 %
glm-5.2	4/5	13/15	10/15	10/15	74 %
qwen3.7-max	4/5	15/15	10/15	12/15	82 %
deepseek-v3.2	1/5	8/15	0/15	2/15	22 %
ministral-14b-2512	2/5	4/15	0/15	2/15	16 %

Table 6.1: Correct diagnoses per difficulty tier and model.

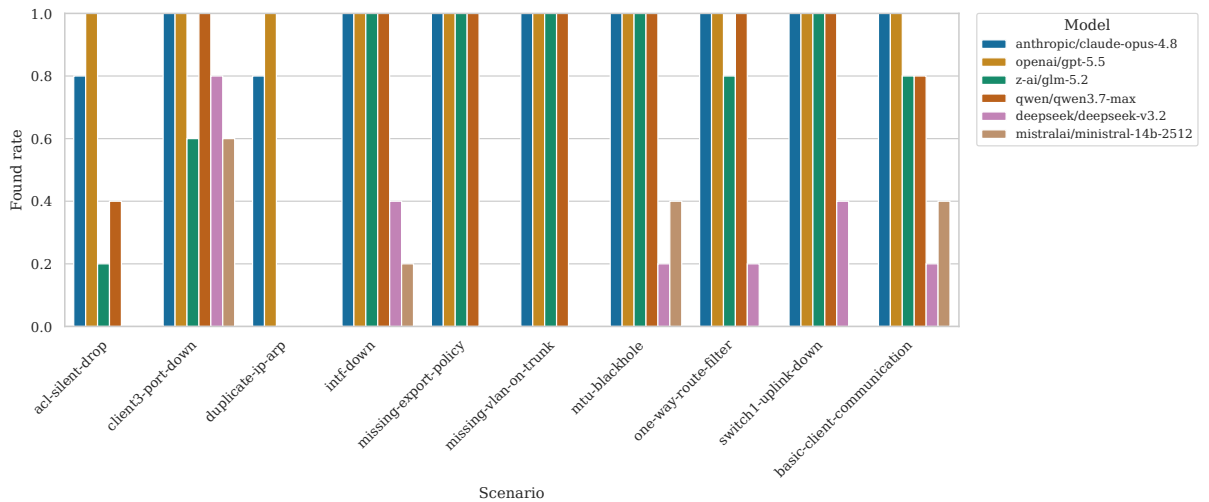


Figure 6.1: Found rate per scenario and model.

The two high-tier models diagnosed 98 of their 100 runs correctly (GPT-5.5 all 50, Claude Opus 4.8 48 of 50). The mid tier reached 74 % (GLM-5.2) and 82 % (Qwen3.7-Max). The low tier reached 22 % (DeepSeek-V3.2) and 16 % (Ministral-14B). Over all six models, the success rate per difficulty tier is 70 % (baseline), 78 % (easy), 54 % (medium) and 61 % (hard).

At scenario level the misses are not spread evenly. `duplicate-ip-arp` was solved only by the high-tier models (GPT-5.5 5/5, Claude Opus 4.8 4/5). Every other model failed it in all five attempts. The scenario `acl-silent-drop` was almost the same (GPT-5.5 5/5, Claude Opus 4.8 4/5, GLM-5.2 1/5, Qwen3.7-Max 2/5, low tier 0/5). These two scenarios also account for the only two misses of Claude Opus 4.8. The `mtu-blackhole` scenario (which was classified as hard) was solved in every run by all four high- and mid-tier models.

One type of failure that only happened with GLM-5.2 is that in 4 of its 50 runs the orchestrator produced a final answer after a single LLM request without calling a single tool. This means that the run ended before any investigation had taken place. All four runs (one each on `acl-silent-drop`, `client3-port-down`, `duplicate-ip-arp` and `one-way-route-filter`) were counted as misses. No other model showed this type of failure. The consequences for GLM-5.2's reported success rate are discussed in [section 7.5](#).

On the healthy baseline scenario, where the correct result is that the network is healthy, the high-tier models were right in all 10 runs, GLM-5.2 and Qwen3.7-Max in 4 of 5 each, DeepSeek-V3.2 in 1 of 5 and Ministral-14B in 2 of 5. The nine wrong baseline runs split into two failure reasons. Six (4 DeepSeek-V3.2, 1 GLM-5.2, 1 Qwen3.7-Max) hit the 600-second cap without delivering any answer (DNF) and Ministral-14B's three wrong results each reported a fault that did not exist.

6.2 Time

The median wall-clock times per run, over all 50 runs of each model, were: Claude Opus 4.8 118s, GPT-5.5 187s, GLM-5.2 248s, Qwen3.7-Max 292s and Ministral-14B 53s. DeepSeek-V3.2’s median equals the 600-second cap because more than half of its runs were stopped due to the time limit. The true durations beyond the cap are unknown. The full wall-clock distribution over all 300 runs, with the stopped runs visible at the cap, is shown in fig. A.3 in the appendix.

Figure 6.2 shows the runs that ended in a correct diagnosis and finished within the time limit. There the medians are: Claude Opus 4.8 114s (48 runs), GPT-5.5 187s (50), GLM-5.2 232s (37), Qwen3.7-Max 266s (41), DeepSeek-V3.2 442s (11) and Ministral-14B 57s (8).

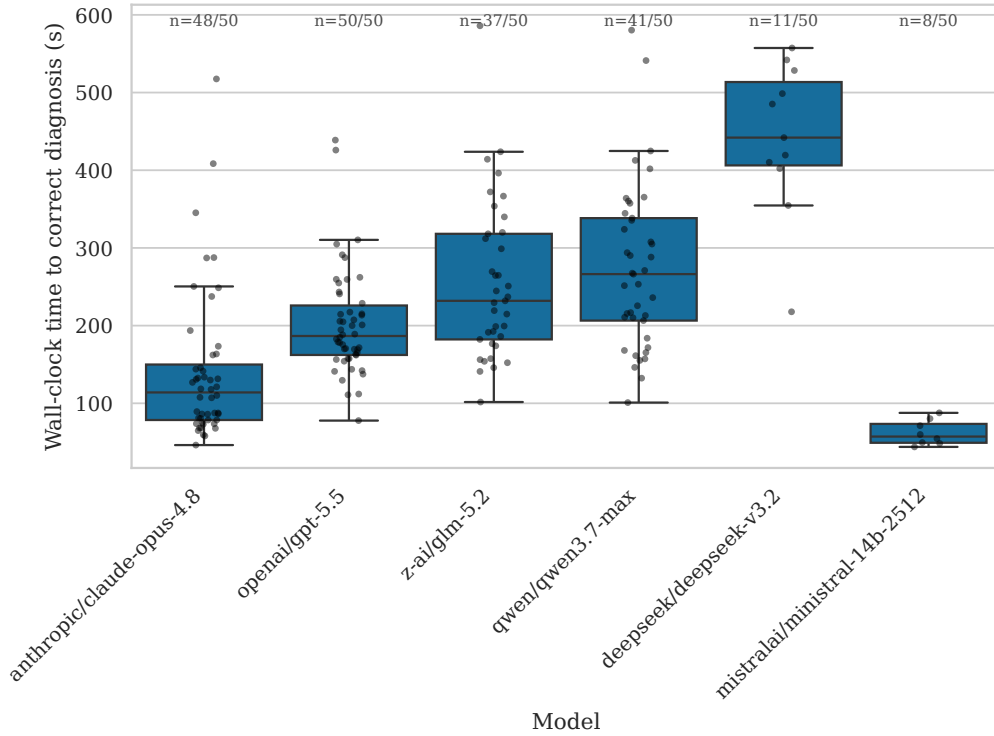


Figure 6.2: Wall-clock time to a correct diagnosis per model.

Interestingly, for a given model the time varies little across the difficulty tiers. Claude Opus 4.8’s median time to a correct diagnosis is 78s on easy and only 118s on hard scenarios, GPT-5.5 needed 170s and 217s. For some models, figuring out that the healthy baseline is healthy took as long as or longer than locating an easy fault (Qwen3.7-Max: 477s versus 172s, though its baseline median rests on only four correct runs). A per-scenario breakdown (showing how much time each model took for a correct diagnosis) is shown in fig. A.2 in the appendix.

Wall-clock time is not the same as LLM inference time. Figure 6.3 shows the LLM inference time summed over all round-trips of a run next to the wall-clock time, over all 50 runs of each model.³ On average this sum exceeds the wall-clock time for every model because the orchestrator agent can launch multiple sub-agents, which then issue LLM requests in parallel. This means overlapping requests are each counted in full. The size of the gap between the two measures indicates how much of a model’s work ran concurrently.

³Summed from the per-request generation times reported by OpenRouter’s Generation API. For the nine runs where the API did not resolve every round-trip (see section 6.3), the missing requests are not part of the sum.

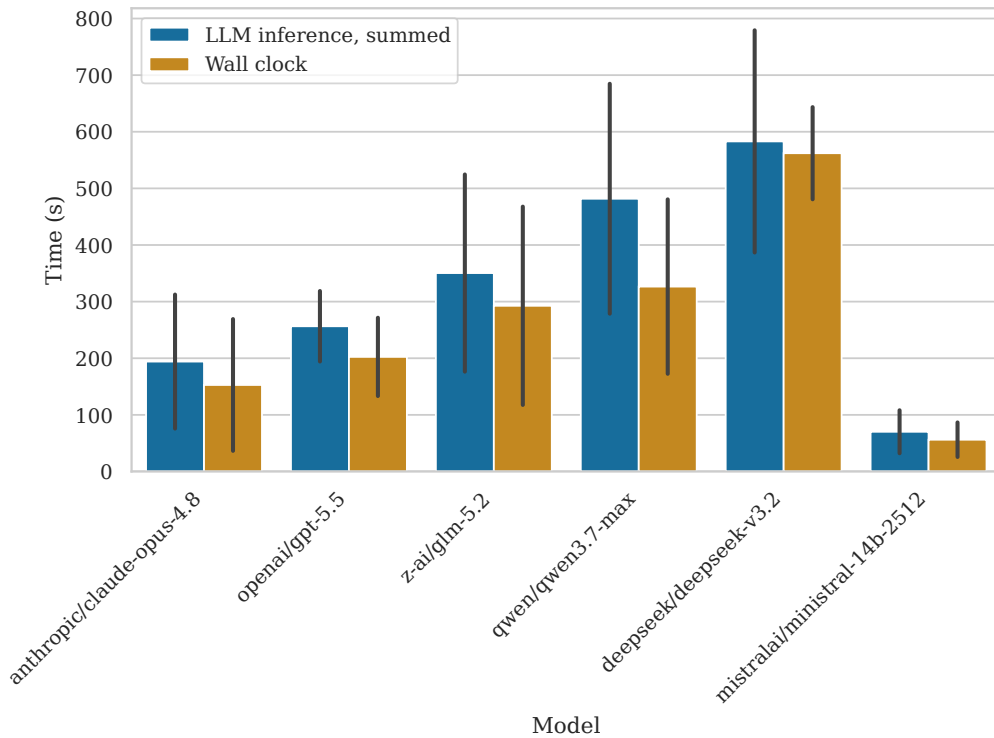


Figure 6.3: Mean LLM inference time per run (summed over all round-trips) and mean wall-clock time per run.

6.3 Cost and Effort

Table 6.2 summarises the main per-model numbers. [fig. 6.4](#) shows mean cost per run against found rate. “Cost/correct” is a model’s total cost divided by its number of correct results. For Ministral-14B this rests on only 8 correct runs. The 300 runs cost \$242 in API fees in total, \$145 of which (60 %) went to GPT-5.5. All costs are OpenRouter list prices at run time. The exact rates used in the experiments are shown in [section A.1](#).

Model	Found rate	Median time to correct (s)	Cost/run (USD)	Cost/correct (USD)	DNF
claude-opus-4.8	96 %	114	1.17	1.22	1
gpt-5.5	100 %	187	2.91	2.91	0
glm-5.2	74 %	232	0.16	0.21	8
qwen3.7-max	82 %	266	0.45	0.55	8
deepseek-v3.2	22 %	442	0.14	0.64	38
ministral-14b-2512	16 %	57	0.01	0.07	0

Table 6.2: Per-model summary. Found rate, cost/run and DNF count are over all 50 runs. Median time to correct is over the correct, finished runs only. Cost/correct is the total cost of all 50 runs divided by the number of correct results.

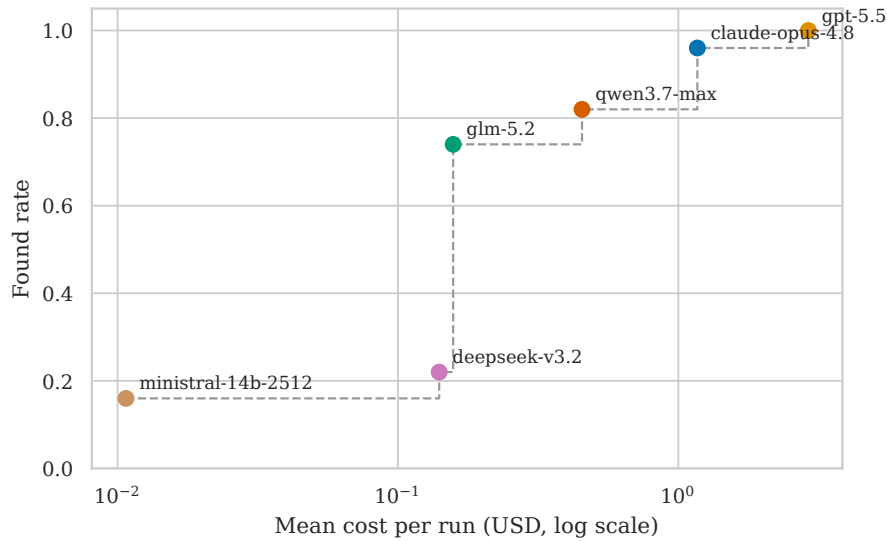


Figure 6.4: Mean cost per run against found rate.

Token usage differs widely across models. Figure 6.5 shows the mean input and output volume per run in *normalised* tokens. OpenRouter re-counts every request with its own model-agnostic tokenizer (“QuadChars”), which the Generation API reports alongside the provider’s native token counts [28]. Unlike the native counts used for billing, these numbers compare how much text each model actually processed before giving a final result, independent of its own tokenizer.⁴ The mean input token count ranges from 42 000 (Minstral-14B) at the low end, through 85 000 (Opus 4.8), 105 000 (GLM-5.2) and 229 000 (Qwen3.7-Max), up to 375 000 (GPT-5.5) and 441 000 (DeepSeek-V3.2). It is notable how little input Opus 4.8 consumed. It reached the second-best found rate on less than a quarter of GPT-5.5’s input tokens.

⁴Normalised counts are missing for 9 of the 300 runs, where the Generation API did not return a resolved record for every LLM round-trip. Those runs are omitted from the means. Native token counts and costs are unaffected.

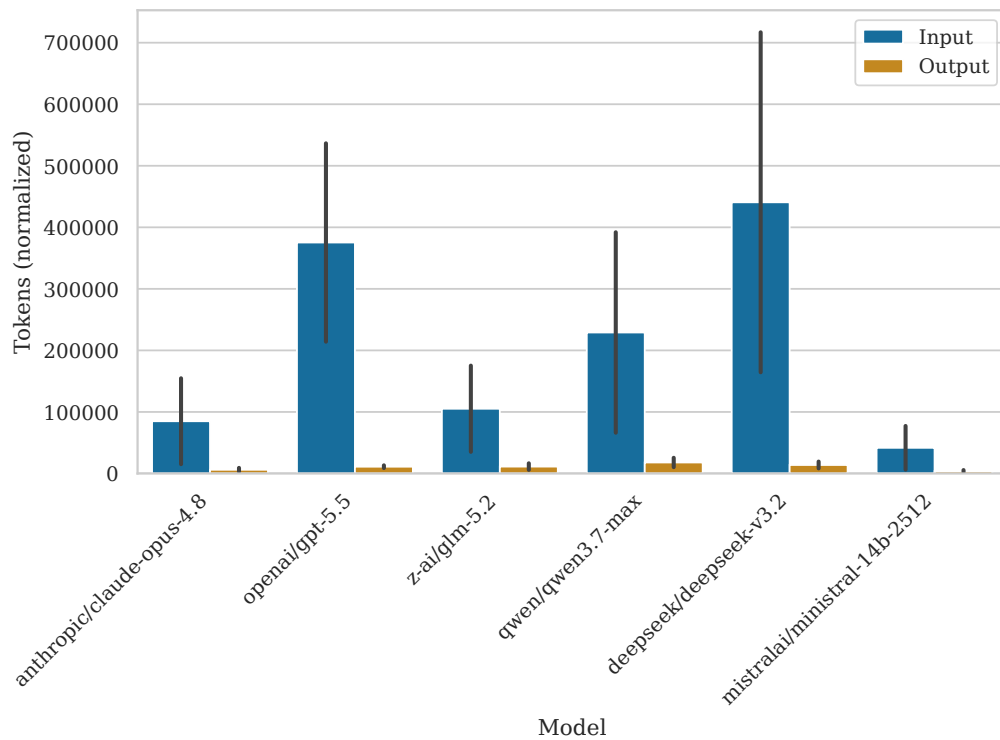


Figure 6.5: Mean normalised input and output tokens per run.

The number of tool calls per run, shown in fig. 6.6, largely mirrors the input-token ranking. The exception is DeepSeek-V3.2, which processed the most input tokens of all models despite a mid-field tool-call count. GPT-5.5 issued by far the most calls (mean 124 per run), followed by Qwen3.7-Max (98), GLM-5.2 and DeepSeek-V3.2 (60 and 59) and Claude Opus 4.8 (46). Ministral-14B stopped after a mean of 24 calls. GPT-5.5 needed about 2.7 times as many tool calls as Opus 4.8 to reach a nearly identical found rate. Ministral-14B’s low count means it did not retrieve much information (by calling tools) before giving a final result.

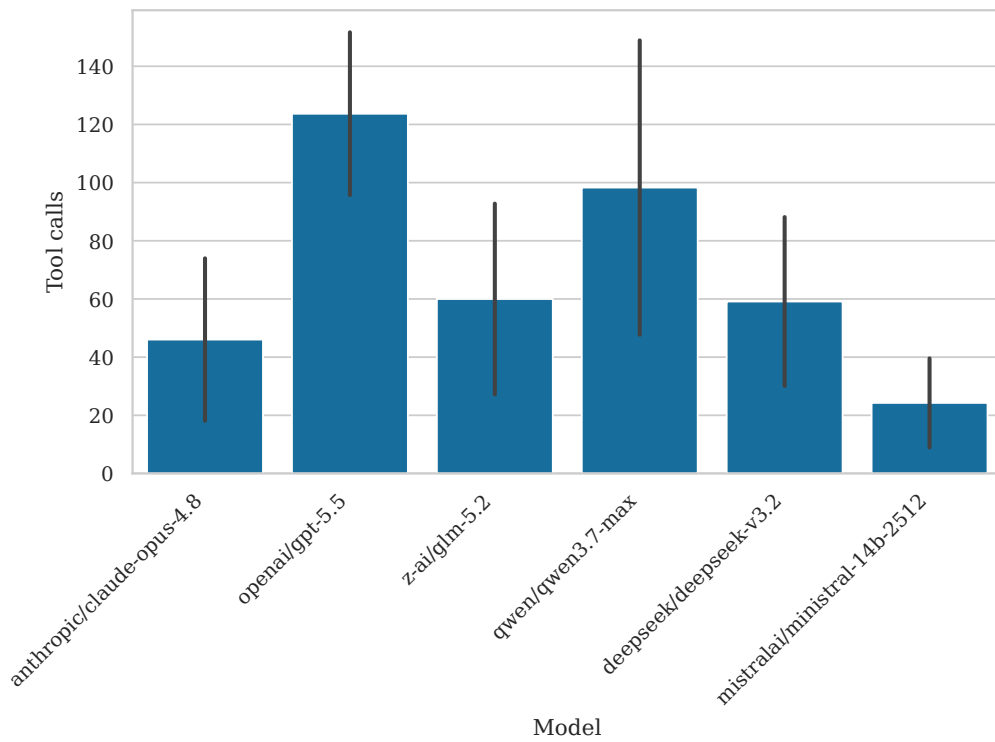


Figure 6.6: Mean number of tool calls per run.

A figure showing the number of invalid commands executed is shown in fig. A.1 in the appendix. Malformed commands were rarest for Ministral-14B and the high-tier models and by far most frequent for Qwen3.7-Max.

7 Evaluation

The results of section 6 are interpreted here with focus on the five research questions from section 1.2. One caveat applies to everything: the test lab is small, with four SR Linux nodes, four clients and only one injected fault at a time. Additionally, each problem is described by only one open prompt. There is no real human interacting (giving directions or generally more information) with the agent system. The results show what the system can do in such an environment. This work does not claim the same results would be achieved for large enterprise networks (see section 7.5).

7.1 Success Rate by Difficulty

Three patterns stand out in the correctness numbers (table 6.1, fig. 6.1).

High tier

With 98 of 100 runs correct, the difficulty tiers for the problems barely matter for GPT-5.5 and Claude Opus 4.8. Hard scenarios such as the MTU blackhole or the one-way route filter were found as reliably as a downed interface. Within this lab, the answer to the difficulty question is that all difficulty tiers from the defined scenarios can be troubleshot, but only the most expensive models handle every tier reliably.

Mid tier

GLM-5.2 and Qwen3.7-Max are reliable on easy faults, handling that tier almost perfectly (28 of 30 runs), but they miss runs on medium and hard scenarios. Their misses concentrate on `duplicate-ip-arp` and `acl-silent-drop`, and most came from running out of time rather than from a wrong conclusion. 16 of their 22 misses are runs that hit the 600-second cap without delivering a final answer and only two ended in an actual wrong diagnosis. (Four of GLM-5.2's misses have a completely different cause, runs that ended without any investigation, see section 7.5.) The nominally hard `mtu-blackhole` scenario was not a problem for them, so the manually assigned difficulty tiers only partially predict what the models actually struggle with. No single characteristic was identified that reliably predicts whether a scenario is truly difficult for an LLM.

Low tier

DeepSeek-V3.2 and Ministral-14B are not usable for this task (network troubleshooting) based on the results, and they fail in different ways. DeepSeek works slowly and often does not finish: 38 of its 50 runs hit the 600-second cap. Ministral finishes quickly but is wrong, which is even worse. It answers within about a minute, yet mostly names the wrong cause or none. Its mean of 24 tool calls per run (fig. 6.6) shows why. It stops gathering information very quickly and bases the final response on little information. The healthy baseline scenario shows the same split: DeepSeek timed out without an answer in 4 of its 5 baseline runs, while Ministral reported a fault that did not exist in 3 of its 5 (section 6.1). In real operation, such false alarms would cost engineer time instead of saving it.

The combined per-tier success rates (70–78 % baseline/easy, 54–61 % medium/hard) average over six models of very different ability, so they mainly reflect the composition of the model set rather than the difficulty tiers themselves. The split by model in table 6.1 is the more informative view, and it shows that the success rate depends far more on the model than on the scenario difficulty.

7.2 Time to Diagnosis

For the time question the relevant metric is the wall-clock time from query to a correct final answer (fig. 6.2). Wrong diagnoses are excluded since they save no time, and someone has

to notice and correct them. By this metric a correct diagnosis takes about two minutes with Claude Opus 4.8, three with GPT-5.5 and four to four-and-a-half with the mid-tier models. Ministral’s one-minute median looks impressive but covers only the 8 runs it got right, and DeepSeek’s correct runs took over seven minutes. The time also barely increases with difficulty. For the high-tier models a hard scenario takes only about 40–50 seconds longer than an easy one (section 6.2).

Part of why the wall-clock time stays this low is the parallelism of the architecture. For every model, the LLM inference time summed over all round-trips exceeds the wall-clock time of the run (fig. 6.3). The orchestrator can launch multiple sub-agents, which can issue their requests concurrently, so more model time is consumed than passes on the clock. Qwen3.7-Max shows the largest gap (a mean of roughly 480 s of summed inference against 325 s of wall clock), meaning it made the most use of parallel sub-agents.

No human control group was used in this project, so the comparison with required manual troubleshooting time is based on an estimate. For the injected faults, a plausible range for a network operator (reproduce the symptom, work through the devices, find the cause) is in the range of 10–15 minutes for the easy tier, 30–60 minutes for the medium tier and about one to two hours for the hard tier. Against that estimate the agent system saves little time on trivial faults (a downed interface is quick to find either way) but, even after adding a few minutes of operator time to read and verify the report, potentially 90 % or more time on medium and hard faults, where its time stays flat at a few minutes while manual effort grows. This is only true when the diagnosis is correct, which for the observed models means the high price tier. At a 74–82 % success rate a network operator would still have to check every answer before acting on it.

A related observation concerns the healthy baseline. For some models, confirming that nothing is wrong took as long as or longer than locating an easy fault (Qwen3.7-Max: 477 s versus 172 s, section 6.2). That makes sense. A fault search can stop at the first plausible cause it finds, whereas declaring a network healthy requires ruling out every possibility. In practice this is very relevant since some problems might not be related to the network and the result should indicate this.

7.3 Cost

Cost is best compared per correct diagnosis, a model’s total spend divided by its number of correct diagnoses (the “cost/correct” column of table 6.2), since a wrong diagnosis has no value. Per correct diagnosis, GLM-5.2 is the cheapest usable option at \$0.21, Claude Opus 4.8 costs \$1.22 and GPT-5.5 \$2.91. DeepSeek-V3.2 illustrates why cost per run is misleading on its own. It is as cheap as GLM-5.2, but because it rarely succeeds, each correct answer effectively costs three times as much. In the cost–success plane of fig. 6.4, the found rate rises monotonically with cost, so every model buys some amount of additional found rate for its extra cost and none is simply worse on both axes than a cheaper one. The two low-tier models are very cheap, but their found rate is too low to be usable.

A saving estimate

As an example we assume the cost of CHF 100 per network operator hour. With the manual-time estimates from [section 7.2](#), diagnosing a medium fault manually costs around CHF 50–100 and a hard one CHF 100–200. The same diagnosis by the agent system costs \$1–3 in API fees with a high-tier model plus a few minutes of network operator time to read and verify the report. This is estimated at CHF 10–15 in total. Even under these assumptions the potential saving per medium or hard incident is roughly a factor of three to twenty, depending on the tier and the assumed manual time. The saving comes mostly from the replaced network operator time. The

choice of model matters little here (in terms of cost), since the API fee is small compared to a single network operator hour either way. This is an illustration rather than an evidence-based measurement, because the operator hours are only estimates.

Price volatility

Two constraints apply to the cost figures. First, they are modelled from token counts and the models' advertised OpenRouter prices at run time and not from OpenRouter's billing system. Provider routing and prompt caching can make the billed amount differ, typically downward because cached prompt tokens are not billed by a provider that applies caching. Caching was therefore excluded deliberately from the cost calculation because its availability varies by provider and model and is often best-effort, which would have made the cross-model comparison depend on routing luck and on whether a model even does implicit caching. Second, OpenRouter prices are volatile (based on provider) especially for open models, where multiple providers compete for the same model. During the window in which the experiments were run, GLM-5.2's input price moved from \$0.95 to \$0.94 and then \$0.93 per million tokens (section A.1). Three days after GLM-5.2's last run, on 6 July 2026, its advertised price was about 27 % lower than what the runs were logged at. The proprietary models (Opus 4.8, GPT-5.5, Qwen3.7-Max) kept a constant list price throughout. The exact rates in effect were recovered from the logs and are listed in section A.1. Repricing the entire experiment at the rates of 6 July 2026 changes the total by under 1 %, so the comparisons above are robust to this change.

7.4 Influence of the Model

Across all metrics, model choice is a bigger factor than scenario difficulty. The found rate spans 16 % to 100 % across the model price tiers, while within one model the difference between easy and hard scenarios is comparatively small. The price tiers from section 3.5 translate into three distinct behaviours. The high tier (Opus 4.8, GPT-5.5) reaches near-perfect success on every difficulty tier, at \$1.22–\$2.91 per correct diagnosis versus \$0.21–\$0.55 for the mid tier, roughly 2–14 times as much depending on the pairing. It also invents no faults on the healthy baseline. The mid tier (GLM-5.2, Qwen3.7-Max) is the cost-effectiveness sweet spot for easy faults, but its 8 DNFs per model are concentrated on `duplicate-ip-arp` (medium tier) and `acl-silent-drop` (hard tier), with a few more spread across the baseline and easy scenarios. In almost all of these misses the model ran out of time rather than answering wrongly. The low tier (DeepSeek-V3.2, Ministral-14B) is not usable: one model is too slow to finish, the other is fast but usually wrong, and both are unreliable on the healthy baseline scenario.

The models also took different paths to their results. GPT-5.5 reaches its perfect found rate with heavy tool use, a mean of 124 tool calls and 375 000 normalised input tokens per run (fig. 6.6, fig. 6.5). Claude Opus 4.8 reaches a nearly identical rate with about a third of the tool calls and less than a quarter of the input tokens. This gap in tokens used is what mainly drives the difference in cost per correct diagnosis (\$2.91 versus \$1.22) – the two models are priced similarly per token, so GPT-5.5 would remain more expensive even at Opus 4.8's token price.

Robustness follows the same ordering with one exception, the number of malformed device commands per run (section 6.3) is lowest for Ministral and the high tier and by far the highest for Qwen3.7-Max. Ministral's low count is partly a consequence of its short runs: a model that issues few commands has few opportunities to use an invalid one. The ability to issue syntactically valid commands and the ability to diagnose correctly appear to be independent.

7.5 Limitations

The results must be considered within the limits of the experiment.

The lab is small and single-vendor: four Nokia SR Linux nodes and four clients, with exactly one injected fault at a time. An enterprise network has many more devices, mixed vendors, concurrent faults and noisy alarms. Nothing here shows how the system scales to that. The results are a feasibility statement for small environments and not for big enterprise networks. The runs are also non-interactive, each consists of a single symptom description with no follow-up dialogue (section 5.4). In a real use case, a network operator could answer questions the agents raise, add observations, or steer the investigation. Such help would plausibly raise the success rate, particularly for the models that stopped at a plausible but wrong explanation. The reported numbers measure the fully autonomous mode.

Three restrictions have to be considered regarding the measurements. The time and cost savings compare against an estimate of how long an engineer would need. No human trials were run. Each scenario-model pair was repeated only five times, so one additional or one fewer successful run shifts a rate by 20 percentage points. And the decision whether a run was a success or a miss was made by one person against a fixed criterion. Borderline answers could be judged differently by another person.

Two further restrictions concern the recorded data. First, GLM-5.2’s success rate may be understated. In 4 of its 50 runs the model ended the agent loop after a single response without calling any tools (the reasoning stated that tools should be called, but none were), so no investigation took place (section 6.1), all four count as misses. Excluding these degenerate runs, its found rate would be 37/46 ($\approx 80\%$) instead of 74 %. GLM-5.2 had been released not long before the experiment window, so issues in its tool-calling layer are a plausible cause. This means the reported 74 % may understate the model’s actual diagnostic ability. Secondly, runs were capped at 600 seconds, and this cap does not weigh equally on all models. The cap also shapes the difficulty answer for the mid tier: 16 of the mid-tier models’ 22 misses are cap hits rather than wrong conclusions, so with a longer time budget their picture on the medium and hard scenarios could look better. Token generation speed differs between models, and for the open-weight models it is partly a property of the serving provider (compute power) rather than of the model itself. A slowly served model reaches the time limit, without being able to do as much work as a fast one, independent of its diagnostic ability. DeepSeek-V3.2 is the clearest case. A large part of its timeouts can be attributed to slow token generation rather than to a deeper investigation, since its tool-call count is close to GLM-5.2’s (fig. 6.6) while far more of its runs hit the cap, so its low found rate partly reflects serving speed instead of only diagnostic capability. Its true time and success numbers beyond the cap are unknown (more than half of its runs were cut off). Token and cost figures for DNF runs are even a bit undercounted. Usage is recorded only when an agent run returns, so the cap discards the orchestrator’s own consumption and that of any sub-agent still working, counting only the sub-agents that had already completed. The reported cost of models with many DNFs understates what they actually consumed.

8 Conclusion and Future Work

8.1 Conclusion

This project built an AI-driven multi-agent system that troubleshoots network problems by splitting and delegating work to specialised sub-agents. This system was used to run 300 experiments. The experiments consisted of a healthy baseline plus nine fault scenarios across three difficulty tiers, using six models from three price tiers, with five runs each. Every run was scored manually against the known root cause. With respect to the research questions from section 1.2, the experiment supports the following answers (within the limitations mentioned in section 7.5).

1. **Time:** With certain models the system delivers a correct result in two to four-and-a-half minutes. Compared to an estimated 10 minutes to 2 hours of manual work depending on the tier, the saving is small for trivial faults but can be large for more difficult problems. The comparison rests on an estimated manual troubleshooting time. No human trials were done.
2. **Difficulty:** Every difficulty tier was solvable, but only the high-tier models solved every tier reliably. Mid-tier models handled the easy tier reliably but failed mostly on two scenarios, the medium-tier `duplicate-ip-arp` and the hard-tier `acl-silent-drop`, by hitting the time cap without a result rather than concluding wrongly. The manually assigned tiers predicted model behaviour only partially.
3. **Success rate:** GPT-5.5 diagnosed 50/50 runs correctly and Claude Opus 4.8 48/50. The mid tier reached 74–82 % overall (near-perfect on easy, about 70 % on medium and hard combined). The low tier models were unusable at 16–22 %.
4. **Cost:** A complete investigation costs \$0.01–\$2.91 in API fees per run depending on the model, or \$0.07–\$2.91 per correct diagnosis (\$0.21–\$2.91 for the mid- and high-tier models that are actually usable). Against an example rate of CHF 100 per network operator hour, a correct automated diagnosis of a medium or hard fault costs a fraction of the manual equivalent.
5. **Model influence:** The model is the deciding factor. With the same agent logic, success ranges from 16 % to 100 % depending only on which model is used, while the API fee per correct diagnosis spans \$0.21–\$2.91 across the usable models.

Beyond the measured experiments, the project’s second part, the automatic syslog investigation (section 3.7), proved feasible. During development the system reacted to real syslog events without any human trigger, opened self-contained investigations, produced initial triage reports and persisted them so that an operator can take over the case later without repeating the completed work (section 4.6). This part was validated qualitatively and was deliberately not part of the quantitative evaluation.

Using a multi-agent system for network troubleshooting is technically feasible today. With a high-tier model, root-cause detection was very good in this lab. Whether this holds up in a production network, with more devices, more alarms and multiple faults, remains open.

8.2 Future Work

The weakest assumption in this evaluation is the estimated manual baseline. A study in which network operators troubleshoot the same faults would replace it with measured data. The same work could also cover an interactive use case of the agent system, where network operators and agents are working together instead of the fully autonomous single-prompt mode measured here. Future evaluations should also use a larger environment: more devices, mixed vendors, several concurrent faults, background noise in syslog, and scenarios where the root cause of the problem lies outside the network (DNS, host configuration).

A second thing is the harness itself. The experiments varied the model while the harness stayed fixed, but the harness is a system of its own. One technique that was not applied in this work is *agent skills*. They package instructions and reference material that an agent discovers through a name and a short description, whose full content is loaded into the context only when the task requires it. This is a principle called *progressive disclosure* [29]. Further harness-side improvements are explicit prompt caching (for models which allow client side steering), which would lower the real API cost per run (the cost metric of section 5.5 deliberately prices uncached tokens). Additionally a finer history compaction that summarises older tool returns instead of replacing them entirely (section 4.5).

Three further experiments follow directly from the findings. The first is a check of the architecture itself. The multi-agent design was never compared against a single agent holding all tools in one context. Running the matrix with such a flattened variant would show whether the decomposition of section 3.2 contributes to the success rate or whether the model would perform equally well without it.

The second is a sensitivity analysis of the wall-clock cap. The mid-tier models failed mostly by hitting the 600-second cap without a result (section 7.5), so their measured success rate depends partly on where the cap sits. Repeating the failed combinations with a higher cap would show better where the limits of these models are.

The third is mixed-model configurations. In the experiments the same model was used across all agents (section 5.3), although the sub-agents work on narrower tasks than the orchestrator. A configuration with a high-tier orchestrator and cheaper sub-agent models was not tested.

→ w! . much privacy implications?

but learn mit dem self hosted

GLM oder so wird schon

was es sich mit dem verhält

9 Auxiliary Tools

The following tools were used in the creation of this work:

- **DeepL write** – To rephrase.
- **Claude Code / Claude AI** – programming, templating, text editing, rephrasing and drafting assistance in parts of the report, grammar and spelling mistakes, research.
- **Google Scholar and Google Scholar Labs** – For research.
- **Consensus AI Research** – For research.

References

- [1] C. Castelfranchi, “Modelling social action for AI agents”, *Artificial Intelligence*, vol. 103, no. 1, pp. 157–182, 1998, Artificial Intelligence 40 years later, ISSN: 0004-3702. DOI: 10.1016/S0004-3702(98)00056-3. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0004370298000563>.
- [2] R. Sapkota, K. I. Roumeliotis, and M. Karkee, “AI agents vs. agentic AI: A conceptual taxonomy, applications and challenges”, *Information Fusion*, vol. 126, p. 103 599, Feb. 2026, ISSN: 1566-2535. DOI: 10.1016/j.inffus.2025.103599. [Online]. Available: <http://dx.doi.org/10.1016/j.inffus.2025.103599>.
- [3] M. R. Douglas, *Large language models*, 2023. arXiv: 2307.05782 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2307.05782>.
- [4] J. Alammar and M. Grootendorst, *Hands-on large language models: language understanding and generation*. O’Reilly Media, Inc., 2024.
- [5] N. F. Liu et al., “Lost in the middle: How language models use long contexts”, *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. DOI: 10.1162/tac1_a_00638. [Online]. Available: <https://aclanthology.org/2024.tac1-1.9/>.
- [6] K. Hong, A. Troynikov, and J. Huber, “Context rot: How increasing input tokens impacts LLM performance”, Chroma, Tech. Rep., Jul. 2025. [Online]. Available: <https://trychroma.com/research/context-rot>.
- [7] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O’Sullivan, and H. D. Nguyen, *Multi-agent collaboration mechanisms: A survey of LLMs*, 2025. arXiv: 2501.06322 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2501.06322>.
- [8] C. Qu et al., “Tool learning with large language models: A survey”, *Frontiers of Computer Science*, vol. 19, no. 8, Jan. 2025, ISSN: 2095-2236. DOI: 10.1007/s11704-024-40678-2. [Online]. Available: <http://dx.doi.org/10.1007/s11704-024-40678-2>.
- [9] M. Wang et al., “Function calling in large language models: Industrial practices, challenges, and future directions”, *ACM Comput. Surv.*, vol. 58, no. 9, Feb. 2026, ISSN: 0360-0300. DOI: 10.1145/3788284. [Online]. Available: <https://doi.org/10.1145/3788284>.
- [10] OpenAI. “Create a model response | OpenAI API reference”, Accessed: Jun. 27, 2026. [Online]. Available: <https://developers.openai.com/api/reference/resources/responses/methods/create>.
- [11] OpenAI. “Function calling | OpenAI API”, Accessed: Jul. 2, 2026. [Online]. Available: <https://developers.openai.com/api/docs/guides/function-calling>.
- [12] S. Yao et al., *ReAct: Synergizing reasoning and acting in language models*, 2023. arXiv: 2210.03629 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2210.03629>.
- [13] Anthropic. “Building effective agents”, Accessed: Jun. 30, 2026. [Online]. Available: <https://www.anthropic.com/engineering/building-effective-agents>.
- [14] Strands Agents. “Agent loop - Strands Agents SDK documentation”, Accessed: Jun. 30, 2026. [Online]. Available: <https://strandsagents.com/docs/user-guide/concepts/agents/agent-loop/>.
- [15] Databricks. “What is an AI agent harness?”, Accessed: Jun. 30, 2026. [Online]. Available: <https://www.databricks.com/blog/ai-harness>.
- [16] V. Trivedy. “The anatomy of an agent harness”, LangChain, Accessed: Jun. 30, 2026. [Online]. Available: <https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>.
- [17] Model Context Protocol. “What is the Model Context Protocol (MCP)?”, Accessed: Jul. 20, 2026. [Online]. Available: <https://modelcontextprotocol.io/docs/getting-started/intro>.

- [18] Anthropic. “Introducing the Model Context Protocol”, Accessed: Jun. 29, 2026. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>.
- [19] Databricks. “What is the Model Context Protocol (MCP)?”, Accessed: Jun. 29, 2026. [Online]. Available: <https://www.databricks.com/blog/what-is-model-context-protocol>.
- [20] Pydantic. “Pydantic AI - agent framework documentation”, Accessed: Jul. 2, 2026. [Online]. Available: <https://pydantic.dev/docs/ai/overview/>.
- [21] OpenRouter. “OpenRouter documentation”, Accessed: Jul. 2, 2026. [Online]. Available: <https://openrouter.ai/docs>.
- [22] FastMCP. “FastMCP documentation”, Accessed: Jul. 2, 2026. [Online]. Available: <https://gofastmcp.com/>.
- [23] Nokia. “JSON-RPC basics - SR Linux learn portal”, Accessed: Jul. 2, 2026. [Online]. Available: <https://learn.srlinux.dev/tutorials/programmability/json-rpc/basics/>.
- [24] Grafana Labs. “Grafana Loki documentation”, Accessed: Jul. 2, 2026. [Online]. Available: <https://grafana.com/docs/loki/latest/>.
- [25] Pydantic. “Pydantic Logfire documentation”, Accessed: Jul. 2, 2026. [Online]. Available: <https://logfire.pydantic.dev/docs/>.
- [26] Containerlab. “Containerlab – container-based networking labs”, Accessed: Jul. 20, 2026. [Online]. Available: <https://containerlab.dev>.
- [27] Nokia. “Learn SR Linux – Nokia SR Linux documentation portal”, Accessed: Jul. 20, 2026. [Online]. Available: <https://learn.srlinux.dev>.
- [28] OpenRouter. “Opus 4.7’s new tokenizer: What it actually costs”, Accessed: Jul. 16, 2026. [Online]. Available: <https://openrouter.ai/blog/insights/opus-47-tokenizer-analysis/>.
- [29] Anthropic. “Equipping agents for the real world with Agent Skills”, Accessed: Jul. 20, 2026. [Online]. Available: <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>.

Declaration of Authenticity

I hereby declare that any individual work / pair work / team work submitted for assessment is entirely the product of my own / my own and my partner's / my own and my team's effort,

- that I/we have correctly cited all text passages that do not originate from me/us, in accordance with standard academic citation rules⁵, and that I/we have clearly mentioned all sources used;
- that I/we have declared in footnotes or in an index of auxiliary tools all aids used (AI assistance systems such as chatbots⁶, translation⁷, paraphrasing⁸, or programming applications⁹);
- that I/we have acquired all intangible rights to any materials I/we may have used, such as images or graphics, or that these materials were created by me/us;
- that the topic, the thesis or parts of it have not been used in an assessment of another module, unless this has been expressly agreed with the lecturer in advance and is stated as such;
- that I/we am/are aware that my/our work may be checked for plagiarism and for third-party authorship of human or technical origin (artificial intelligence);
- that I/we am/are aware that the FHNW School of Engineering will pursue a violation of this declaration of authenticity and that disciplinary consequences (reprimand or expulsion from the study program) may result from this.

Windisch, July 20, 2026

Name: Dominik Krebs

Signature:

⁵e.g. APA oder IEEE

⁶e.g., ChatGPT

⁷e.g., DeepL

⁸e.g., Quillbot

⁹e.g., Github Copilot

A Appendix

A.1 Model Prices During the Experiment Window

The per-run costs in the experiment log are computed as native token counts times the model’s advertised OpenRouter price at run time. Table A.1 lists the rates in effect. Only GLM-5.2 changed price during the window. The open-weight models (GLM-5.2, DeepSeek-V3.2, Ministral-14B) are served by multiple competing providers on OpenRouter, so their advertised price is whatever the cheapest provider currently charges and can move at any time.

Model	Period	Input \$/Mtok	Output \$/Mtok
anthropic/claude-opus-4.8	whole window	5.00	25.00
openai/gpt-5.5	whole window	5.00	30.00
qwen/qwen3.7-max	whole window	1.25	3.75
deepseek/deepseek-v3.2	whole window	0.2288	0.3432
mistralai/ministral-14b-2512	whole window	0.20	0.20
z-ai/glm-5.2	2026-06-29	0.95	3.00
z-ai/glm-5.2	2026-06-30	0.94	3.00
z-ai/glm-5.2	2026-07-02 – 07-03	0.93	3.00

Table A.1: OpenRouter list prices per million tokens during the experiment window.

A.2 Additional Experiment Figures

Figure A.1 shows the malformed device commands as totals over the five runs of each scenario–model pair. Figure A.2 breaks the time to a correct diagnosis down to scenario level (correct, completed runs only). A missing row means the model never solved that scenario. Figure A.3 shows the wall-clock time of all 300 runs, including wrong diagnoses and DNFs. The runs stopped by the time limit form the clusters at the 600-second cap.

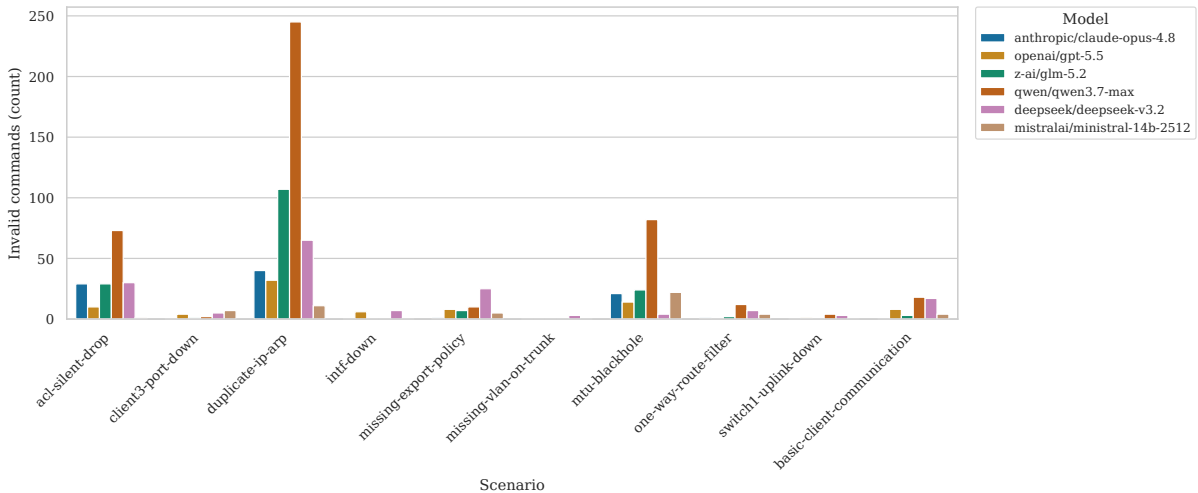


Figure A.1: Malformed device commands per scenario and model.

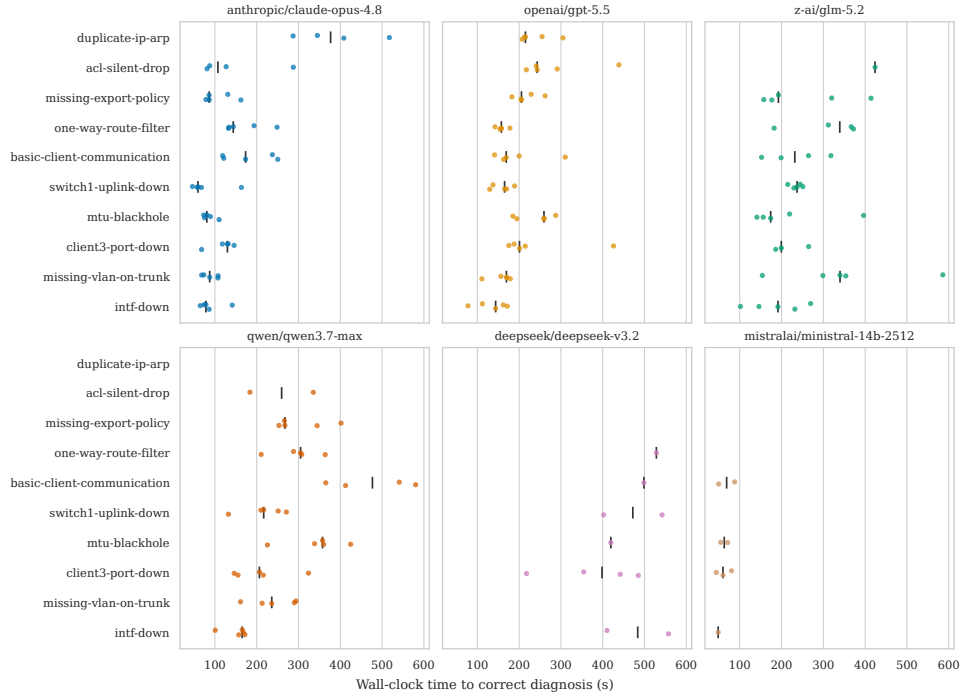


Figure A.2: Wall-clock time to a correct diagnosis per scenario and model.

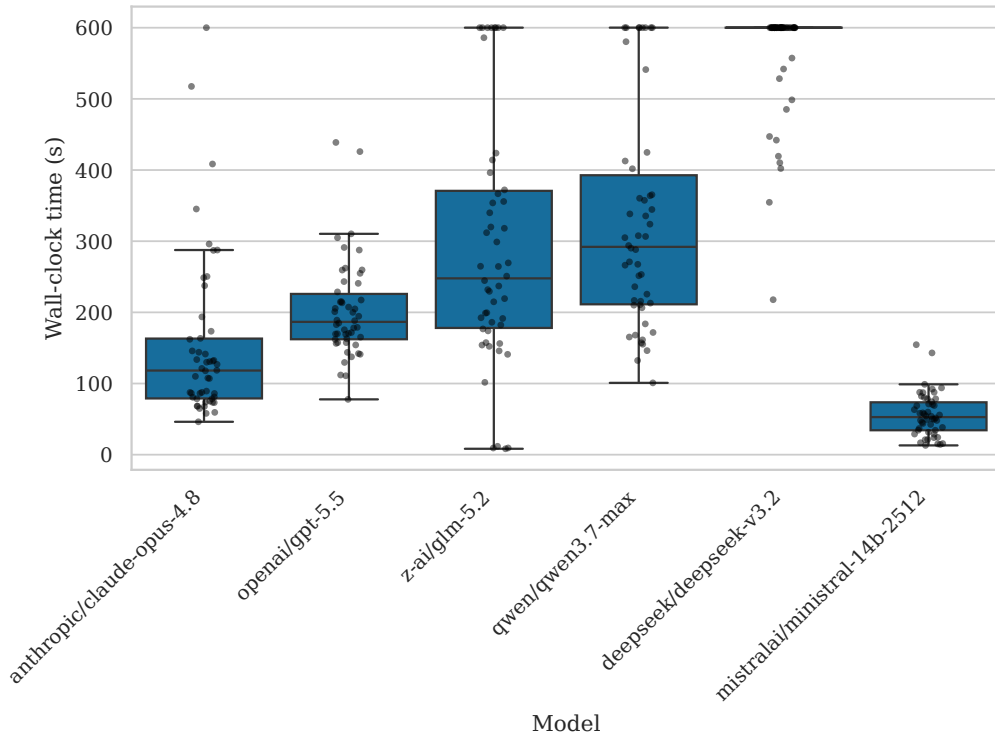


Figure A.3: Wall-clock time per run over all 300 runs, including wrong diagnoses and DNFs. Boxes show median and interquartile range. Dots are individual runs. The clusters at 600s are the 55 runs stopped at the wall-clock cap.